LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Naufal Samih Najasyi
NIM : 2428240116
Kelas : SI5B

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

Identitas Praktikum

Nama Mahasiswa	Naufal Samih Najasyi
NPM	2428240116
Kelas	SI5B
Pertemuan Ke-	5
Topik / Judul Praktikum	Implementasi RESTful API Menggunakan Express.js
Tanggal Pelaksanaan	24
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1.
Memahami konsep dasar RESTful API dan penerapan routing pada Express.js.

2.
Menerapkan endpoint HTTP untuk mengambil, menambah, dan memperbarui data mahasiswa

3.
Melakukan pengujian endpoint API menggunakan server lokal dan memastikan respons berjalan sesuai fungsi yang dibuat.

II. Dasar Teori

RESTful API
RESTful API merupakan antarmuka layanan yang memanfaatkan resource dan metode HTTP untuk berkomunikasi antara client dan server. Dalam Express.js, routing didefinisikan melalui method pada objek aplikasi seperti app.get() dan app.post(), sehingga setiap request dapat diarahkan ke handler sesuai endpoint dan metode HTTP yang digunakan. (Express.js, n.d.) Pada praktikum ini resource yang digunakan adalah data mahasiswa melalui endpoint /mahasiswa. GET digunakan untuk mengambil data, termasuk pengambilan berdasarkan ID dan penyaringan menggunakan query parameter. POST digunakan untuk mengirim data baru ke server, sedangkan PUT digunakan untuk memperbarui data pada resource tertentu.

Semantik metode-metode tersebut sesuai dengan standar HTTP yang dijelaskan MDN Web Docs. (MDN Web Docs, 2025)
Data praktikum disimpan sementara pada array di dalam aplikasi. Oleh karena itu, perubahan data selama pengujian berlaku selama proses server berjalan dan tidak menggunakan database permanen.

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	v24.21.0
2	Visual Studio Code	Editor kode dan terminal
3	npm	v11.19.0
4	Git	v2.56.0.windows.1
5	Vercel	Platform web (vercel.com) — deployment serverless
6	Github	Platform web (github.com) — repository hosting

IV. Langkah Kerja

1.

Memastikan Node.js dan npm terpasang serta terminal berada pada folder project C:\SI5B_Express\restful-api-app.

2.

Memeriksa source code Express.js dan menemukan penggunaan cors() yang belum memiliki import/dependency.

3.

Menjalankan npm install cors dan menambahkan const cors = require('cors'); pada app.js.

4.

Menjalankan server menggunakan perintah npm start hingga muncul pesan Server berjalan di http://localhost:4000.

5.

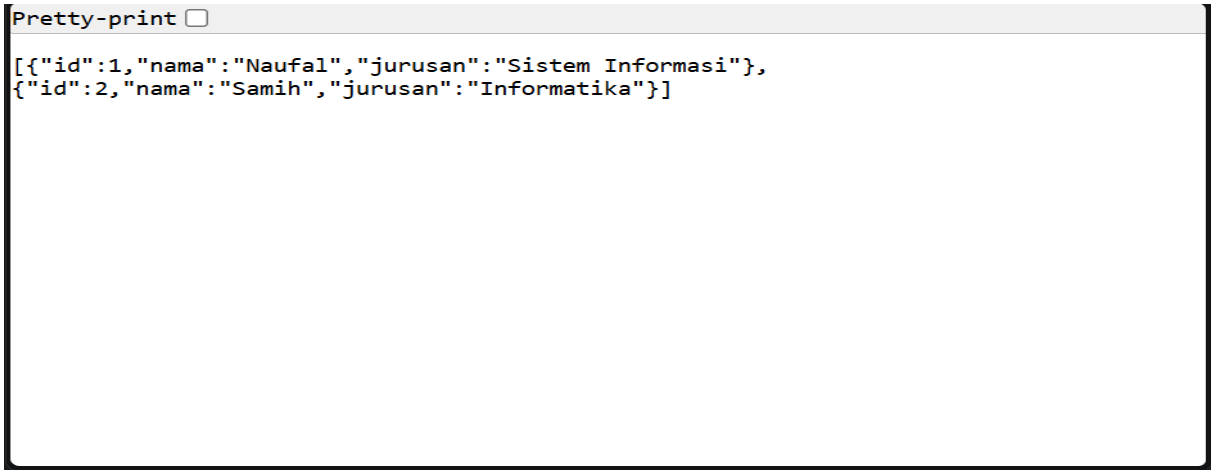
Menguji endpoint RESTful API secara berurutan menggunakan request lokal, meliputi GET seluruh data, GET berdasarkan ID, GET dengan filter jurusan, POST data mahasiswa, dan PUT data mahasiswa.

Endpoint yang diuji pada praktikum:

- GET /mahasiswa
- GET /mahasiswa/:id
- GET /mahasiswa?jurusan=Informatika
- POST /mahasiswa
- PUT /mahasiswa/:id

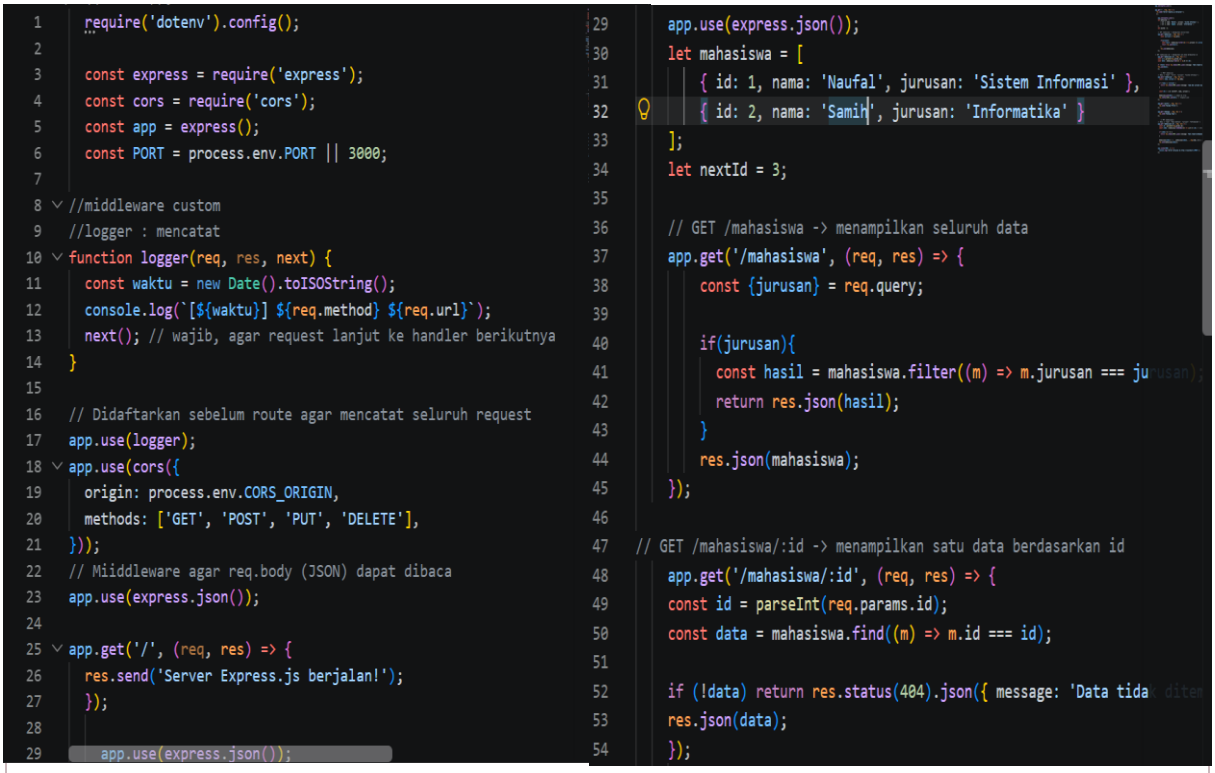
V. Hasil dan Pembahasan

Setelah dependency cors dilengkapi dan server dijalankan, aplikasi berhasil berjalan pada localhost port 4000. Pengujian dilakukan terhadap endpoint yang telah tersedia pada project asli. Hasil utama pengujian ditampilkan pada tabel berikut.



Gambar 1. Hasil GET /mahasiswa pada server local

Metode	Endpoint	Status	Hasil
GET	/mahasiswa	200 OK	2 data awal berhasil ditampilkan
GET	/mahasiswa/1	200 OK	Data mahasiswa ID 1 berhasil ditampilkan
GET	/mahasiswa?jurusan=Informatika	200 OK	Filter menghasilkan data jurusan Informatika
POST	/mahasiswa	201 Created	Data Citra berhasil ditambahkan
PUT	/mahasiswa/2	200 OK	Data ID 2 berhasil diperbarui



```
require('dotenv').config();

const express = require('express');
const cors = require('cors');
const app = express();
const PORT = process.env.PORT || 3000;

//middleware custom
//logger : mencatat
function logger(req, res, next) {
  const waktu = new Date().toISOString();
  console.log(`[${waktu}] ${req.method} ${req.url}`);
  next(); // wajib, agar request lanjut ke handler berikutnya
}

// Didaftarkan sebelum route agar mencatat seluruh request
app.use(logger);
app.use(cors({
  origin: process.env.CORS_ORIGIN,
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
}));
// Miiddleware agar req.body (JSON) dapat dibaca
app.use(express.json());

app.get('/', (req, res) => {
  res.send('Server Express.js berjalan!');
  });

    app.use(express.json());
  app.use(express.json());
  let mahasiswa = [
      { id: 1, nama: 'Naufal', jurusan: 'Sistem Informasi' },
      { id: 2, nama: 'Samih', jurusan: 'Informatika' }
  ];
  let nextId = 3;

  // GET /mahasiswa -> menampilkan seluruh data
  app.get('/mahasiswa', (req, res) => {
      const {jurusan} = req.query;

      if(jurusan){
        const hasil = mahasiswa.filter((m) => m.jurusan === ju
        return res.json(hasil);
      }
      res.json(mahasiswa);
  });

// GET /mahasiswa/:id -> menampilkan satu data berdasarkan id
  app.get('/mahasiswa/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const data = mahasiswa.find((m) => m.id === id);

  if (!data) return res.status(404).json({ message: 'Data tida
  res.json(data);
  });
```

Gambar 2. Source Code Endpoint RESTful API pada File app.js

Pembahasan:

Pengujian menunjukkan bahwa endpoint GET dapat mengambil seluruh data mahasiswa maupun data tertentu berdasarkan ID. Query parameter jurusan juga dapat digunakan untuk menyaring data. Pada operasi POST, server menghasilkan status 201 Created dan memberikan ID baru untuk data yang ditambahkan. Pada operasi PUT, server mengembalikan status 200 OK setelah data dengan ID tertentu diperbarui. Hasil tersebut menunjukkan bahwa fungsi dasar pengambilan, penambahan, dan pembaruan resource sudah berjalan pada aplikasi Express.js.

Middleware CORS ditambahkan agar aplikasi dapat menerima request dari origin yang diizinkan berdasarkan konfigurasi CORS_ORIGIN pada file environment. Parsing JSON menggunakan express.json() memungkinkan data pada request body dibaca oleh handler POST dan PUT.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	Perintah node dan npm pada awalnya belum dikenali oleh PowerShell.	Memasang Node.js dan membuka kembali terminal, kemudian memverifikasi dengan node -v dan npm -v.
2	Server gagal dijalankan karena fungsi cors() digunakan tetapi module cors belum diimport/dipasang.	Menjalankan npm install cors dan menambahkan const cors = require('cors'); pada app.js.

VII. Kesimpulan

Praktikum berhasil menghasilkan RESTful API sederhana menggunakan Express.js. Setelah dependency dan import CORS diperbaiki, server dapat dijalankan pada localhost port 4000. Endpoint GET, GET berdasarkan ID, filter menggunakan query parameter, POST, dan PUT berhasil diuji sesuai fungsi masing-masing. Praktikum juga memberikan pemahaman mengenai routing Express.js, request/response HTTP, penggunaan JSON body, dan pengujian API secara langsung.

Daftar Pustaka

Express.js. (n.d.). Routing. Express.js. https://expressjs.com/en/guide/routing.html

MDN Web Docs. (2025, July 4). HTTP request methods. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods

Node.js. (n.d.). Download Node.js®. https://nodejs.org/en/download/

Lampiran

Tautan Repository GitHub:

https://github.com/naufal20102006/si5b_latihan_restful_naufal

Tautan Deployment (jika ada):

si5b-latihan-restful-naufal - Overview – Vercel